

DS : Assignment 1A

1. What is a data structure?

Data structure is a systematic way to organize data.

It enables efficient storage and retrieval of information. It defines the relationships between data elements.

It underpins the performance of algorithms.

2. What is an Array? Give examples for Integer, String, and Character types.

An array is a collection of elements stored contiguously.

It holds items of the same data type in a fixed-size structure. For integers: [1, 2, 3, 4].

For strings: ["apple", "banana", "cherry"] and characters: ['a', 'b', 'c'].

3. What is indexing in an Array?

Indexing is the process of accessing elements by their position. It starts at zero in most programming languages.

Each element is referenced by its index.

This provides direct access and efficient retrieval.

4. Mention advantages of arrays in specific use cases.

Arrays allow constant time access to elements using indexes. They use contiguous memory, enhancing cache performance. They are simple to implement and understand.

They are efficient for static collections of data.

5. What are the different types of data structures?

Data structures can be linear or non-linear.

Linear types include arrays, linked lists, stacks, and queues. Non-linear types include trees and graphs.

They each serve specific purposes for data organization.

6. How does a data type differ from a data structure?

A data type defines the nature of data (e.g., int, float).

A data structure organizes and stores multiple data items. Data types are the building blocks for data values.

Structures provide the framework for managing those values.

7. What are primitive and non-primitive data types?

Primitive types are basic building blocks like int, char, and float.

They are directly supported by the programming language.

Non-primitive types are collections or composites, such as arrays or objects.

They are constructed from primitive data types.

8. Give an example of a mutable and an immutable data type in Python.

Mutable types can be changed after creation. Examples include lists and dictionaries.

Immutable types cannot be altered after creation. Examples include strings and tuples.

9. What is the difference between an array and a linked list?

Arrays have fixed sizes and store elements contiguously.

Linked lists use nodes connected by pointers with dynamic sizes. Arrays provide fast random access using indices.

Linked lists excel in dynamic insertion and deletion.

10. **Why is choosing an appropriate data structure important?** The right data structure optimizes time and space efficiency. It simplifies problem-solving and coding.

It directly impacts performance in software applications. It ensures maintainability and scalability of solutions.

11. **What are linear and non-linear data structures?** Linear structures store data in sequential order. Examples include arrays, stacks, and queues.

Non-linear structures arrange data hierarchically or in graphs. Examples include trees and graphs.

12. Define static and dynamic data structures with examples.

Static data structures have fixed sizes defined at compile time. Arrays are typical examples of static structures.

Dynamic data structures can change size during runtime. Linked lists and dynamic arrays are dynamic examples.

13. What is the role of Python's built-in list in data structures?

Python's list is a dynamic array implementation. It supports resizing, insertion, and deletion.

It offers flexible storage for various data types. It is widely used for its simplicity and efficiency.

14. What is memory allocation?

Memory allocation is reserving space in memory for data storage. It occurs during compile time (static) or runtime (dynamic).

It ensures that programs have the required memory to execute. Efficient allocation is key for performance optimization.

15. Differentiate between static and dynamic memory allocation.

Static allocation reserves memory at compile time with fixed size. Dynamic allocation reserves memory during program execution.

Static is faster but less flexible, while dynamic is flexible but overhead exists.

Each serves different needs based on application requirements.

16. How does Python manage memory internally?

Python uses an automatic memory management system. It employs a private heap for all object allocations.

Garbage collection reclaims memory from unused objects.

This helps prevent memory leaks and manage resources efficiently.

17. What are memory leaks and how can they be avoided?

Memory leaks occur when allocated memory isn't freed after use. They gradually degrade performance and resource availability.

Proper object dereferencing and garbage collection help avoid them. Using memory management best practices is essential.

18. What is the difference between stack and heap memory?

Stack memory is used for static memory allocation during function calls. It is organized in a last-in, first-out (LIFO) manner.

Heap memory is used for dynamic memory allocation at runtime.

Heap is larger, less structured, and managed manually or automatically.

19. How does recursion affect memory usage?

Recursion uses additional stack frames for each function call.

Each recursive call consumes extra memory until the base case is met. Excessive recursion can lead to stack overflow errors.

Efficient recursion design minimizes memory overhead.

20. What is recursion?

Recursion is a function calling itself to solve smaller instances. It breaks a problem into similar, simpler sub-problems.

A base case terminates the recursive calls.
It provides a clear, elegant solution for many problems.

21. How does recursion work in Python?

A recursive function calls itself with reduced input parameters. It continues until it reaches a base case that stops recursion.
Each call adds a new layer to the call stack.
Python manages these calls until all resolve and return.

22. What is the base case in recursion?

The base case is the condition where recursion stops. It prevents infinite loops by terminating further calls. It provides a direct answer without further recursion. Every recursive function must define a clear base case.

23. How can infinite recursion be avoided?

Infinite recursion is prevented by defining a proper base case. Each recursive call must progress towards that base case.
Careful logic and conditions ensure termination.
Testing and debugging help confirm proper recursive behavior.

24. Write a simple recursive function to find the factorial of a number.

```
def factorial(n): if n
    == 0:
        return 1
    else:
        return n * factorial(n-1)

num = int(input("Enter positive number: ")) # Enter
positive number: 7
print(f"The factorial of {num} is {factorial(num)}") # The
factorial of 7 is 5040
```

25. What is time complexity?

Time complexity measures how the running time increases with input size.
It provides a high-level understanding of algorithm efficiency. Big-O notation is used to express worst-case performance.
It helps in comparing and optimizing algorithms.

26. What is space complexity?

Space complexity evaluates the extra memory an algorithm needs. It measures how storage requirements grow with input size.

Both time and space complexity impact overall performance. Understanding it guides efficient algorithm design.

27. What is asymptotic analysis?

Asymptotic analysis examines algorithm behavior for large inputs. It focuses on growth rates rather than exact values.

Big-O, Theta, and Omega notations are used in this analysis. It provides insights into long-term performance trends.

28. Define best case, worst case, and average case complexities.

Best case is the minimal time an algorithm takes for favorable inputs. Worst case is the maximum time required under the most challenging conditions.

Average case is the expected time over all possible inputs. These metrics guide algorithm selection and optimization.

29. Explain the importance of Big-O notation.

Big-O notation describes the upper bound of an algorithm's performance.

It simplifies complexity analysis by focusing on dominant terms. It helps compare algorithms regardless of hardware differences. Big-O is fundamental for evaluating scalability and efficiency.

30. What is Theta (Θ) notation?

Theta notation provides a tight bound on an algorithm's complexity. It represents both upper and lower limits simultaneously.

It precisely characterizes the growth rate of an algorithm.

It is used when the performance is bounded from both sides.

31. How is Big-O notation different from Theta notation?

Big-O gives an upper limit on algorithm performance.

Theta notation gives an exact growth rate by bounding both sides. Big-O is more general, while Theta is precise.

They are both key tools for complexity analysis.

32. What is a stack?

A stack is a linear data structure following LIFO order. Elements are added and removed from the same end.

It is used to manage function calls and undo operations. Its operations are simple and efficient.

33. What are the main operations on a stack?

The primary operations are push, pop, and peek. Push adds an element to the top of the stack.

Pop removes the top element from the stack.

Peek retrieves the top element without removal.

34. How can a stack be implemented using a list in Python?

A Python list can function as a stack.

Use the `append()` method to push an element.

Use the `pop()` method to remove the last element. This mimics the LIFO behavior of a stack.

35. What is a queue?

A queue is a linear data structure following FIFO order.

Elements are added at the rear and removed from the front. It models real-world scenarios like customer service lines.

Queues ensure fair order in processing elements.

36. What are the main operations on a queue?

The main operations are enqueue, dequeue, and front/peek. Enqueue adds an element to the back of the queue.

Dequeue removes an element from the front.

Peek retrieves the first element without removal.

37. How can a queue be implemented using an array?

An array can represent a queue with index management. Use two indices to track the front and rear positions.

Circular arrays help efficiently utilize space.

This maintains the FIFO order for processing elements.

38. What is a priority queue?

A priority queue is a structure where elements are dequeued by priority. Each element is associated with a priority level.

It does not follow the strict FIFO order.

It is used when processing order is based on importance.

39. What is the difference between a stack and a queue?

A stack uses a last-in, first-out (LIFO) order.

A queue uses a first-in, first-out (FIFO) order.
Stacks manage nested operations like recursion.
Queues handle sequential tasks such as scheduling.

40. Explain infix, postfix, and prefix notation with an example.

In infix notation, operators are written between operands ($A + B$). Postfix notation places operators after operands ($A B +$).

Prefix notation places operators before operands ($+ A B$).

These notations affect how expressions are parsed and evaluated.

DS : Assignment 1A

1. What is a singly linked list?

A **singly linked list** is a linear data structure where each node contains:

- **Data** (stores the actual value)
- **Pointer (next)** (stores the address of the next node in the

sequence) The last node points to None, indicating the end of the list.

2. How is memory allocated in a linked list?

Memory is **dynamically allocated** when a new node is created. Python's garbage collector handles memory deallocation automatically.

Example:

```
node = Node(5) # Memory allocated dynamically
```

3. Describe the process of inserting a node at the beginning of a linked list.

- Create a new node.
- Set its next pointer to the current head.
- Update the head pointer to the new node.

Example:

```
new_node.next =
```

```
head head =
```

4. What is the time complexity of searching in a linked list?

- **Worst-case:** $O(n)$ (traverse the entire list)
 - **Best-case:** $O(1)$ (if the element is at the head)
-

5. How can you implement a stack using a linked list?

A stack (LIFO) can be implemented using a linked list by:



Push operation: Insert at the beginning.

- **Pop operation:** Remove from the beginning.
-

6. What are the advantages of a linked list-based stack?

- **Dynamic Size:** No fixed limit like arrays.
 - **Efficient Insertions/Deletions:** Both push and pop run in $O(1)$.
 - **No Memory Wastage:** Uses only the required memory.
-

7. How is a queue implemented using a linked list?

A queue (FIFO) is implemented with:

- A **front pointer** (first element).
- A **rear pointer** (last element).

Operations:

- **Enqueue:** Insert at the rear.
 - **Dequeue:** Remove from the front.
-

8. Explain the process of deleting a node from a linked list.

- Find the node to be deleted.
- Update the previous node's next pointer to skip the node.
- Free the memory of the deleted node.

Example:

```
def delete_node(self, key):  
    temp = self.head  
  
    if temp is not None:  
        if temp.data == key:  
            self.head = temp.next  
            temp = None  
  
    return
```

```
if temp.data == key:
    break

prev = temp
temp =
temp.next

if temp is None:
    return
```

9. What is polynomial addition using linked lists?

Polynomial terms are represented as nodes with:

- **Coefficient**
- **Exponent**

Addition involves merging two polynomials by adding terms with the same exponent.

Algorithm for Polynomial Addition Using Linked Lists

1. **Initialize pointers** to the heads of both polynomials.
2. **Traverse both lists** simultaneously:
 - If **p1.exp > p2.exp**, add p1 to result and move p1 forward.
 - If **p1.exp < p2.exp**, add p2 to result and move p2 forward.
 - If **exponents are equal**, add coefficients, insert the sum, and move both pointers.
3. **Insert remaining terms** from p1 or p2.

10. How are sparse matrices represented using linked lists?

A **sparse matrix** contains mostly zero values. It is represented using a linked list by storing only non-zero elements along with their row and column indices.

Each node contains:

- **Row index**

- **Column index**
 - **Value**
 - **Pointer to the next node**
-

11. What is a doubly linked list?

A **doubly linked list** is a linked list where each node has:

- **Data**
- **Pointer to the next node** (next)
- **Pointer to the previous node** (prev)

This allows traversal in both directions.

12. How do you insert a node in a doubly linked list?

- Create a new node.
- Adjust the prev and next pointers of the surrounding nodes.

Example (Insert at the beginning):

```
def add_at_head(self, new_data):
```

```
    new_node = Node(new_data)
```

```
    new_node.next = self.head
```

```
    if self.head:
```

```
        self.head.prev = new_node
```

13. What is dynamic storage management?

Dynamic storage management refers to allocating and deallocating memory at runtime, allowing efficient use of memory.

In Python, this is handled by:

- **Heap memory allocation** for objects.

- **Garbage collection** for unused objects.
-

14. Describe a circular linked list.

A **circular linked list** is a linked list where:

- The last node points to the first node instead of None.
- This allows continuous traversal in a loop.

It can be **singly** or **doubly** linked.

15. How do you detect a cycle in a linked list?

The Floyd's Cycle Detection Algorithm (Tortoise and Hare) is used:

- Use two pointers: slow moves one step, fast moves two steps.
- If they meet, a cycle exists.

Example:

```
def has_cycle(head):  
    slow, fast = head, head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
    if slow == fast:  
        return True
```

16. How do you define a node in a singly linked list in Python?

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```



17. Write Python code to create a singly linked list with three nodes.

```
head = Node(1)
second = Node(2)

head.next =
second
```

18. How do you insert a node at the beginning of a singly linked list?

- Create a new node.
- Set its next pointer to the current head.
- Update the head pointer.

Example:

```
def add_at_head(self, new_data):
    new_node = Node(new_data)
    new_node.next = self.head
    self.head = new_node
```

19. What is the time complexity of traversing a singly linked list?

- **Worst-case:** $O(n)$ (visit all nodes).
- **Best-case:** $O(1)$ (head contains required data).

20. What is the time complexity of inserting a node at the end of a singly linked list?

- **If the tail pointer is available:** $O(1)$
 - **If only head is available:** $O(n)$ (must traverse to the last node)
-

21. What is the time complexity of deleting a node from a singly linked list?

- **Best case** (delete head): $O(1)$
 - **Worst case** (delete last or search first): $O(n)$
-

22. How do you reverse a singly linked list?

- Initialize three pointers: prev = None, current = head, and next = None.
- Traverse the list, reversing the next pointer of each node.
- Update head to the last processed node.

Time Complexity: $O(n)$

Example:

```
def reverse(self):  
    prev = None  
    curr = self.head  
    while curr:  
        next_node = curr.next # Store next node  
        curr.next = prev # Reverse the link  
        prev = curr # Move prev forward  
        curr = next_node # Move curr forward  
    self.head = prev # Update head to the new first node
```

23. What is the advantage of using a linked list over an array?

- **Dynamic size** (no need to predefine size).
 - **Efficient insertions/deletions** (no shifting required).
 - **Better memory utilization** (no wasted space like arrays).
-

24. What is the disadvantage of using a linked list over an array?

- **More memory usage** (extra pointer per node).
- **Slower access time** (no direct indexing, $O(n)$ search).

- **Cache inefficiency** (nodes are scattered in memory).
-

25. How do you implement a stack using a linked list in Python?

- Use a linked list where the **top** of the stack is the head node.
 - **Push:** Insert at the beginning.
 - **Pop:** Remove from the beginning.
-

26. What is the time complexity of pushing an element onto a linked stack?

- **$O(1)$** (insertion at the head)
-

27. What is the time complexity of popping an element from a linked stack?

- **$O(1)$** (removal from the head)
-

28. How do you implement a queue using a linked list in Python?

- Maintain two pointers: front (head) and rear (tail).
 - **Enqueue:** Insert at the rear.
 - **Dequeue:** Remove from the front.
-

29. What is the time complexity of enqueueing an element in a linked queue?

- ☐ Since we maintain a rear pointer, insertion happens in constant time.
 - ☐ No shifting of elements is required like in an array.
 - **$O(1)$** (insertion at the rear)
-

30. What is the time complexity of dequeuing an element from a linked queue?

- ☐ Since we maintain a front pointer, we simply move it to the next node.
 - ☐ No need to shift elements, making it an efficient operation.
 - **$O(1)$** (removal from the front)
-

31. How do you represent a polynomial using a linked list?

Each term of the polynomial is stored in a node with:

- **Coefficient** (value of the term)
- **Exponent** (power of the variable)
- **Pointer to the next node**

Example

```
def print_poly(self):  
    temp = self.head  
    while temp:  
        print(f"{temp.coeff}x^{temp.exp}", end=" + ") temp  
        = temp.next  
    print("None")
```

output:

Polynomial:

32. What is the time complexity of adding two polynomials using linked lists?

- Both polynomials are traversed once.
- Merging two sorted lists takes **$O(n+m)$** , where n and m are the number of terms in each polynomial.

 Time Complexity: $O(n+m)$

33. What is the difference between a singly linked list and a doubly linked list?

Feature	Singly Linked List	Doubly Linked List
Pointers	One (next)	Two (next & prev)
Memory Usage	Less	More (extra pointer)
Traversal	One direction	Both directions
Insertion/Deletion	Slower (must track previous node)	Faster (direct prev access)

34. How do you insert a node at the end of a doubly linked list?

1. Create a new node.
2. Traverse to the last node.
3. Set `temp.next = new_node` and `new_node.prev =`

`temp(last)` Example:

```
def add_at_tail(self, new_data):  
    new_node = Node(new_data)  
  
    if not self.head: # If list is empty, new node becomes the head  
        self.head = new_node  
  
    return  
  
    temp = self.head  
  
    while temp.next: # Traverse to the last node  
        temp = temp.next
```

35. What is the time complexity of deleting a node from a doubly linked list?

- **Searching for the node:** $O(n)$
- **Deleting the node:** $O(1)$ (adjust next and prev pointers)

 Overall: $O(n)$ in worst case

36. What is the advantage of a circular linked list over a singly linked list?

- **No Null Pointers:** The last node connects back to the first, reducing None checks.
 - **Efficient Traversal:** Any node can be used as a starting point.
 - **Good for Round-Robin Scheduling** (like CPU scheduling).
-

37. How do you create a circular linked list in Python?



1. Create a node.
2. Set `last.next = head` to make it circular.

Algorithm for Creating a Circular Linked List

1. **Create a new node** with given data.
2. **If the list is empty:**
 - Set `head = new_node`.
 - Point `new_node.next` to itself (circular link).
3. **If the list is not empty:**
 - Traverse to the last node.
 - Set `last.next = new_node`.
 - Set `new_node.next = head` to complete the circle.

■ **Time Complexity:** $O(1)$ for empty list, $O(n)$ for insertion at the end.

38. What is the time complexity of traversing a circular linked list?

- Traversal is similar to a singly linked list.
- Stop when we reach the starting node again.

■ Time Complexity: $O(n)$

39. What is garbage collection in Python?

Garbage collection is Python's **automatic memory management** that:

- Frees memory occupied by objects **no longer in use**.
- Uses **reference counting** and **cyclic garbage collection**.

■ Prevents memory leaks.

40. What is the role of the gc module in Python?

The gc module provides:

- **Manual garbage collection control**
- **Checking memory leaks**

- Disabling/enabling automatic garbage collection
-

The End